Assignment Report

On

Analysis of Sorting Algorithms

Submitted by

263CS002 Mukul Rajput

262IS031 Sujal Gupta

Dept. of Computer Science,
NITK, Surathkal

Department of Information Technology
National Institute of Technology Karnataka,
Surathkal

Contents

Introduction **3**

1 Comparison of Sorting Algorithms **4**

1.1 Bubble Sort 4

1.2 Insertion Sort 4

1.3 Merge Sort 5

1.4 Quick Sort 5

1.5 Heap Sort 5

1.6 Radix Sort 5

2 Data Generation and Experimental Setup **5**

2.1 Choice of Test Data 6

2.2 Experimental Setup 6

2.3 Stabilizing Measurements 7

3 Comparison of Quick Sort Versions **8**

3.1 Best Case Running Time 8

3.2 Worst Case Running Time 9

3.3 Average Case Running Time 10

3.4 Conclusion 11

4 Detailed Analysis of Individual Sorting Algorithms **11**

4.1 Bubble Sort 11

4.2 Insertion Sort 12

4.3 Merge Sort 13

4.4 Quick Sort 14

4.5 Heap Sort 15

4.6 Radix Sort 15

5 Comparison of Sorting Algorithms **16**

5.1 Best Case Running Time 17

5.2 Worst Case Running Time 18

5.3 Average Case Running Time 19

5.4 Conclusion 19

6 Correlation of Comparisons with Execution Time 20

6.1 Insertion Sort 20

6.2 Bubble Sort (Optimized) 21

6.3 Heap Sort 21

6.4 Merge Sort 22

6.5 Quick Sort (Median-of-Three Pivot) 23

6.6 Radix Sort 23

6.7 Conclusion 24

7 Execution Time Analysis 24

7.1 Bubble Sort 25

7.2 Insertion Sort 25

7.3 Merge Sort 25

7.4 Heap Sort 26

7.5 Radix Sort 26

7.6 Quick Sort (Median-of-Three Pivot) 26

7.7 Quick Sort: Comparison of the Three Versions 27

7.8 Quick Sort (First Pivot) 27

7.9 Quick Sort (Random Pivot) 28

Overall Conclusion 28

Abstract

This report presents an experimental analysis of six fundamental sorting algorithms: Bubble Sort, Insertion Sort, Merge Sort, Quick Sort, Heap Sort, and Radix Sort. Each algorithm was implemented in C++ and run on randomly generated, increasing, and decreasing integer arrays, and both the execution time and the number of comparisons were recorded for every run. Input sizes range from 10,000 to 500,000 elements. Bubble Sort and Insertion Sort were run up to 200,000 elements, and Quick Sort with a first-element pivot was run on ordered input up to 75,000 elements. Three versions of Quick Sort, based on first-element, random, and median-of-three pivot selection, are compared. For the first-element rule, an additional balanced input, constructed so that the first element is a good pivot, was also recorded.

The results show clear differences between theoretical expectations and measured behavior. On decreasing input of 75,000 elements, first-pivot Quick Sort takes 965.2950 ms, about 241 times its time on random input. The median-of-three implementation also grows quadratically on decreasing input, reaching 10.741 seconds at 500,000 elements against 5.4151 ms on increasing input. Random-pivot Quick Sort never exceeds 31.0128 ms at 500,000 elements. Merge Sort and Heap Sort grow smoothly regardless of the input order, and Radix Sort, which uses no comparisons, is the fastest algorithm on random input at every size, taking 11.7244 ms at 500,000 elements.

Introduction

Sorting is one of the most common operations in computing. It underlies searching, database indexing, scheduling, and many other algorithms, and it is a standard vehicle for teaching algorithm analysis. Textbook analysis describes the running time of a sorting algorithm with asymptotic notation such as $O(n^2)$ or $O(n \log n)$. Such bounds describe how the cost grows with input size, but they hide constant factors, memory behavior, and the effect of the input order. In practice these hidden factors can determine which algorithm is the better choice for a given task.

The aim of this assignment is to compare the theoretical expectations for six classic sorting algorithms with what is actually measured when they are implemented and run. The study has the following objectives:

- to measure the running time of each algorithm on random, increasing, and decreasing inputs of different sizes;
- to compare three pivot-selection strategies for Quick Sort and see how sensitive each is to the input order;
- to compare all algorithms in best-case, worst-case, and average-case settings;
- to examine how strongly the number of comparisons explains the observed execution time; and
- to report the measured averages in tables so that every conclusion can be traced back to recorded data.

The rest of the report is organized as follows. Section 1 summarizes the algorithms and their theoretical behavior. Section 2 describes the test data and the experimental procedure. Section 3 compares the three Quick Sort variants. Section 4 analyzes each algorithm individually, and Section 5 compares the algorithms with one another. Section 6 relates comparison counts to execution time, and Section 7 lists the complete tables of average execution times. The report ends with overall conclusions and a discussion of limitations.

1 Comparison of Sorting Algorithms

This section gives a brief overview of the six algorithms and the principle behind each. The table below summarizes the standard theoretical running times against which the experimental results are later compared.

Algorithm	**Best case**	**Average case**	**Worst case**	**Extra space**
Bubble Sort (optimized)	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$
Radix Sort	$O(d(n+k))$	$O(d(n+k))$	$O(d(n+k))$	$O(n+k)$

Theoretical time and space complexity of the algorithms studied (d is the number of digit positions and k the size of the digit alphabet).

1.1 Bubble Sort

Bubble Sort repeatedly compares adjacent elements and swaps them whenever they are in the wrong order. After each pass the largest remaining element has moved to its final position at the end of the unsorted part, and the process continues until the list is sorted. Its running time is quadratic in general, because up to n passes of up to $n-1$ comparisons are needed. The optimized version used here stops as soon as a complete pass makes no swap, so an already sorted array is processed in a single pass.

1.2 Insertion Sort

Insertion Sort builds a sorted list one element at a time. Each element is taken from the unsorted portion and inserted into its correct position within the sorted portion, shifting larger elements one place to the right. The amount of work depends strongly on how far elements are from their final positions: an already sorted array needs only about $n-1$ comparisons, while a reversed array requires the maximum number of shifts.

1.3 Merge Sort

Merge Sort uses a divide-and-conquer strategy. It splits the array into two halves, sorts each half recursively, and then merges the two sorted halves into one sorted sequence. The recursion has $\log n$ levels and each level does linear work, which gives $O(n \log n)$ time in every case. The price is the auxiliary array needed for merging.

1.4 Quick Sort

Quick Sort partitions the list around a pivot element, placing smaller elements on one side and larger elements on the other, and then applies the same procedure recursively to the two parts. When the pivot splits the array into roughly equal parts the running time is $O(n \log n)$, but when the pivot is repeatedly the smallest or largest element the recursion becomes very unbalanced and the running time degrades to $O(n^2)$. The choice of pivot therefore matters a great deal. This report evaluates three pivot strategies:

- **Pivot Choice 1:** the first element is used as the pivot.
- **Pivot Choice 2:** a randomly chosen element is used as the pivot.
- **Pivot Choice 3:** the median of the first, middle, and last elements is used as the pivot.

1.5 Heap Sort

Heap Sort first converts the list into a heap data structure. It then repeatedly extracts the maximum element and places it at the end of the list, which produces ascending order. Both heap construction and the repeated extractions are bounded by $O(n \log n)$, so the comparison-based running time is $O(n \log n)$ for every input order, and the sort needs no additional array.

1.6 Radix Sort

Radix Sort processes the digits of the numbers one position at a time, using a stable internal procedure for each digit. It does not depend on pairwise comparisons between elements, so its running time depends mainly on the number of elements and the number of digits rather than on the order of the data.

2 Data Generation and Experimental Setup

This section describes how the test data were produced and how the performance of the algorithms was measured. The same basic input categories were used for all algorithms.

2.1 Choice of Test Data

To evaluate each algorithm thoroughly, three types of input were used:

- **Sorted in increasing order:** arrays that are already sorted in ascending order.
- **Sorted in decreasing order:** arrays sorted in descending order.
- **Randomly generated elements:** arrays containing randomly generated integers.

For first-pivot Quick Sort, a fourth input type was also recorded, which is called the *balanced input* in this report. It is an arrangement of the keys for which the first element is a good pivot at every partition step, so that the first-element rule can split the array evenly each time. Its comparison counts are identical, at every size, to those of median-of-three Quick Sort on increasing input, which is consistent with both reaching the same balanced partitioning.

The recorded input sizes are 10,000, 25,000, 50,000, 75,000, 100,000, 200,000, and 500,000 elements. Not every algorithm was run on every size and input type, so the coverage is stated explicitly:

- Merge Sort, Heap Sort, Radix Sort, random-pivot Quick Sort, and median-of-three Quick Sort: all seven sizes and all three input types.
- Bubble Sort and Insertion Sort: sizes from 10,000 to 200,000 (six sizes) and all three input types; there are no measurements at 500,000 elements.
- First-pivot Quick Sort: random input and balanced input at all seven sizes, but increasing and decreasing input only at 10,000, 25,000, 50,000, and 75,000 elements.

All statements in this report refer only to recorded measurements. No value has been estimated or extrapolated for a combination that was not run.

Throughout the report the terms "best case", "worst case", and "average case" are used in an empirical sense. For a given algorithm and input size, the *best case* is the smallest measured average time among the recorded input types, the *worst case* is the largest among the random, increasing, and decreasing inputs, and the *average case* is the time on random input. For first-pivot Quick Sort the best case therefore includes the balanced input, and its worst case is available only up to 75,000 elements, because beyond that size no increasing or decreasing input was recorded and the largest recorded value would be the random input, which is not a worst case.

2.2 Experimental Setup

The experiments were implemented in C++. Execution time was measured with the `std::chrono` timing facility and is reported in milliseconds. For each run, the recorded data contain the algorithm name, input type, input size, round number, execution time, comparison count, and a flag stating whether the output was correctly sorted. This flag is set for all 489 recorded runs.

- **Timing unit:** milliseconds.
- **Input sizes:** 10,000 to 500,000 elements, as listed in Section 2.1.
- **Input types:** random, increasing, and decreasing (plus the balanced input for first-pivot Quick Sort).
- **Algorithms:** Bubble, Insertion, Merge, three Quick Sort versions, Heap, and Radix.

All experiments were run on a single laptop, whose specifications are listed below. The algorithms are implemented as C++ programs that run on the CPU.

Component	Specification
Machine	ASUS TUF Gaming F17 (FX706HF)
Processor	11th Gen Intel Core i5-11400H @ 2.70 GHz
Installed RAM	16 GB (15.7 GB usable), 3200 MT/s
Graphics	NVIDIA GeForce RTX 2050 (4 GB), Intel UHD Graphics
Operating system	Windows 11 Home (version 25H2, OS build 26200.9550), 64-bit

Test machine used for all experiments.

2.3 Stabilizing Measurements

Each experiment was repeated to reduce the effect of small timing fluctuations, and every reported value is the arithmetic mean over the recorded rounds.

- **Repetitions:** every combination of algorithm, input type, and input size has exactly three recorded rounds, which gives 163 combinations and 489 runs in total. All averages in the tables use these three observations.
- **Comparison counts:** for every algorithm except random-pivot Quick Sort, the comparison count is identical in all three rounds, because the input and the pivot choices are the same. For random-pivot Quick Sort the count varies slightly between rounds (for example, 148121, 158782, 156395 at 10,000 random elements), and the table-level analysis uses the mean over the three rounds.
- **Consistent inputs:** the comparison is based on the recorded input type and size, so that the observed difference is associated with the algorithm and pivot strategy rather than with a change in the basic experimental category.

Because each average uses only three rounds, one slow round can shift a mean noticeably. The clearest examples are the increasing-input runs of Heap Sort at 100,000 elements (5.1175, 9.3987, and 6.1112 ms) and of Radix Sort at the same size (3.4385, 2.203, and 4.553 ms).

3 Comparison of Quick Sort Versions

The performance of Quick Sort depends strongly on the choice of pivot. This section compares the three pivot-selection strategies using the measured execution times:

- Pivot Choice 1: first element,
- Pivot Choice 2: random element,
- Pivot Choice 3: median of the first, middle, and last elements.

The analysis considers the best-case, worst-case, and average-case views of the measurements.

3.1 Best Case Running Time

In the best-case comparison, the time of each Quick Sort version is taken as the smallest of its recorded input types at each input size. For the first-element pivot this includes the balanced input described in Section 2.1. For the random-pivot and median-of-three versions the smallest time is obtained on increasing input at every size.

The first-pivot rule has the lowest best-case time at every size, from 0.0779 ms at 10,000 elements to 5.1010 ms at 500,000 elements. Median-of-three follows closely on increasing input (5.4151 ms at 500,000 elements), and the random-pivot version is slower (12.1330 ms at 500,000 elements). The random choice cannot reach the perfectly balanced splits of the other two rules: at 500,000 elements random-pivot Quick Sort makes about 1.35 times as many comparisons on increasing input as median-of-three does (median-of-three makes 8,475,732).

The best case of the first-pivot rule is reached only on the specially constructed balanced input. On random input the same rule takes 29.2294 ms at 500,000 elements, about 5.7 times longer, and its balanced-input time is only about 16% to 19% of its random-input time across the recorded sizes.

The important observation is that pivot selection changes the partition quality. When the pivot produces balanced partitions, the recursive subproblems stay small and the running time follows the expected near-$O(n \log n)$ behavior. When the pivot repeatedly creates highly unbalanced partitions, the number of comparisons grows much faster, as the next subsection shows.

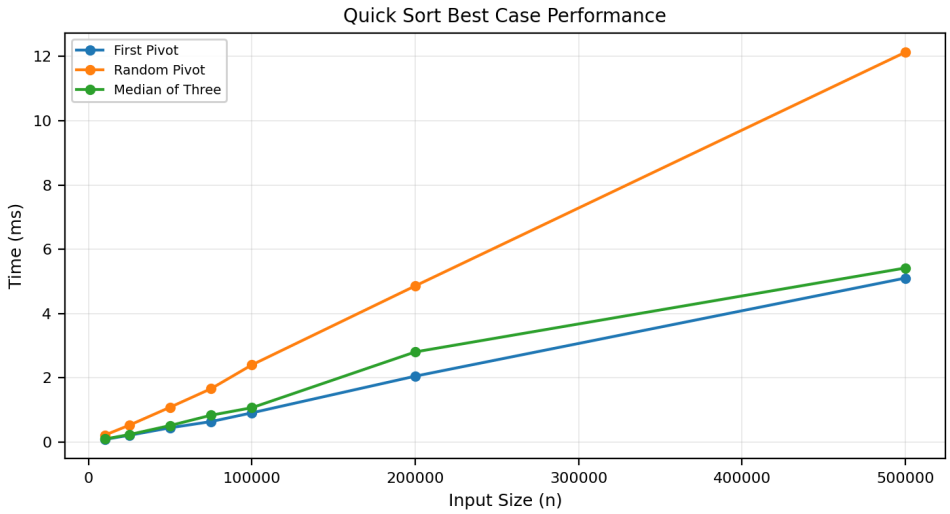


Figure 1: Best Case Running Time of the Three Quick Sort Versions (the first-pivot curve uses the balanced input).

3.2 Worst Case Running Time

The worst-case comparison uses the largest measured time among the random, increasing, and decreasing inputs for each input size. For the first-element pivot this comparison is possible only up to 75,000 elements, where ordered input was recorded. There it shows the expected quadratic behavior: at 75,000 elements the average time is 642.8610 ms for increasing input and 965.2950 ms for decreasing input. These are about 160 and 241 times its random-input time of 4.0119 ms, and about 1,527 times its balanced-input time of 0.6323 ms. When the size grows from 10,000 to 75,000 elements (a factor of 7.5), the time on increasing and decreasing input grows by factors of 51.8 and 51.0, close to the factor of 56.25 expected for quadratic growth. Both ordered inputs make exactly $n(n-1)/2$ comparisons, which is 2,812,462,500 at 75,000 elements. Since no ordered first-pivot runs were recorded beyond 75,000 elements, no value for larger sizes is stated here.

A notable experimental result is that the median-of-three implementation also deteriorates strongly on decreasing input. Its average time rises from 4.6651 ms at 10,000 elements to 10,740.6667 ms at 500,000 elements. The input grows by a factor of 50, while the time grows by a factor of 2,302, close to the factor of 2,500 expected from purely quadratic growth. The comparison counts confirm this: at 500,000 elements the implementation makes 31,250,250,002 comparisons, about 0.125001 n^2. At the same size its decreasing-input time is about 344 times its random-input time and about 1,983 times its increasing-input time. The report therefore does not assume that a named pivot strategy is automatically safe; the actual implementation and its partition behavior matter.

Among the three versions, random-pivot Quick Sort is the most stable. Its worst case is the random input at every size, and it reaches only 31.0128 ms at 500,000 elements.

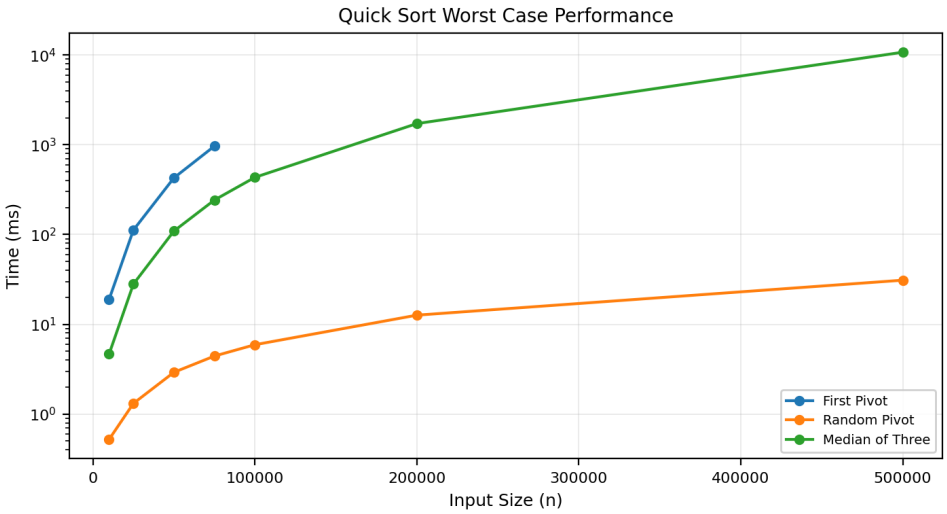


Figure 2: Worst Case Running Time of the Three Quick Sort Versions (logarithmic time axis; the first-pivot curve ends at 75,000 elements, the largest size with recorded ordered input).

3.3 Average Case Running Time

For the average-case view, randomly generated input is used. On random input all three Quick Sort versions scale far better than in their pathological ordered-input cases. At 500,000 elements the measured averages are 29.2294 ms for the first pivot, 31.0128 ms for the random pivot, and 31.1902 ms for the median-of-three pivot.

The first-pivot version is the fastest at every recorded size on the random datasets used in this experiment. This does not contradict the theoretical average-case analysis of Quick Sort, which predicts $O(n \log n)$ time for all three strategies. The first-pivot rule has no extra work for choosing the pivot, while the random and median-of-three rules pay a small constant overhead per partition. The differences among the three are modest: the slowest version is between 2.5% and 24.1% slower than the fastest, depending on the size, and the random-pivot and median-of-three versions swap places from one size to another. Practical running time also depends on the actual input, the pivot sequence, implementation constants, and machine conditions.

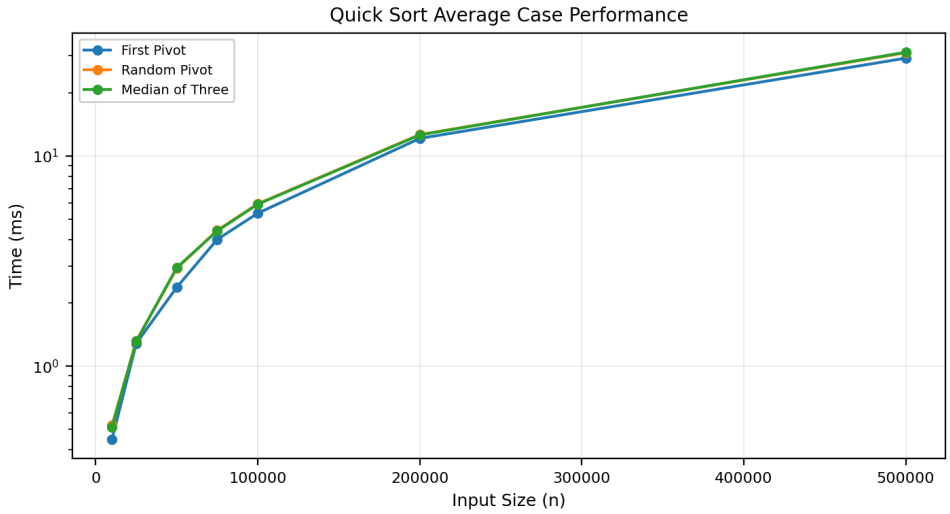


Figure 3: Average Case Running Time of the Three Quick Sort Versions on Random Input (logarithmic time axis).

3.4 Conclusion

The experimental results show that pivot selection has a major effect on Quick Sort. The first-element pivot is very fast on the recorded random data and, on the balanced input, is the fastest of all the versions, but it shows a clear quadratic pattern on increasing and decreasing data. The random-pivot version avoids the large ordered-input blow-up seen in the measurements, because its choice of pivot does not depend on the arrangement of the data. The median-of-three version is very fast on increasing input and performs well on random input, but its decreasing-input results show that a particular implementation should not be assumed to be robust for every ordering.

4 Detailed Analysis of Individual Sorting Algorithms

The following subsections analyze each algorithm separately. For each one, performance was studied under increasing, decreasing, and random input wherever the recorded data contain the corresponding measurement.

4.1 Bubble Sort

Bubble Sort shows the expected quadratic behavior whenever many elements have to be exchanged. Increasing input is processed extremely fast, in 0.0050 ms at 10,000 elements and 0.1101 ms at 200,000 elements, because the optimized implementation terminates after a single pass without swaps and makes only $n - 1$ comparisons. For a 20-fold increase in size the time on increasing input grows by a factor of 21.9, which is linear growth.

On random input the time grows from 157.5460 ms at 10,000 elements to 82,874.8667 ms at 200,000 elements, a factor of 526 against the factor of 400 expected for quadratic growth, so the growth is slightly steeper than quadratic. On decreasing input the time grows from 252.6447 ms to 101,244.0000 ms (about 101.244 seconds), a factor of 400.7, almost exactly quadratic.

Decreasing input is slower than random input at every size (by a factor of 1.60 at 10,000 elements and 1.22 at 200,000 elements). The two inputs need almost the same number of comparisons at 10,000 elements (49,978,710 for random and 49,995,000 for decreasing), so the difference comes from the swaps: a reversed array needs the maximum number of exchanges, about twice as many as a random array on average. Figure 4 uses a logarithmic time axis because the times for increasing input and for the other two inputs differ by several orders of magnitude.

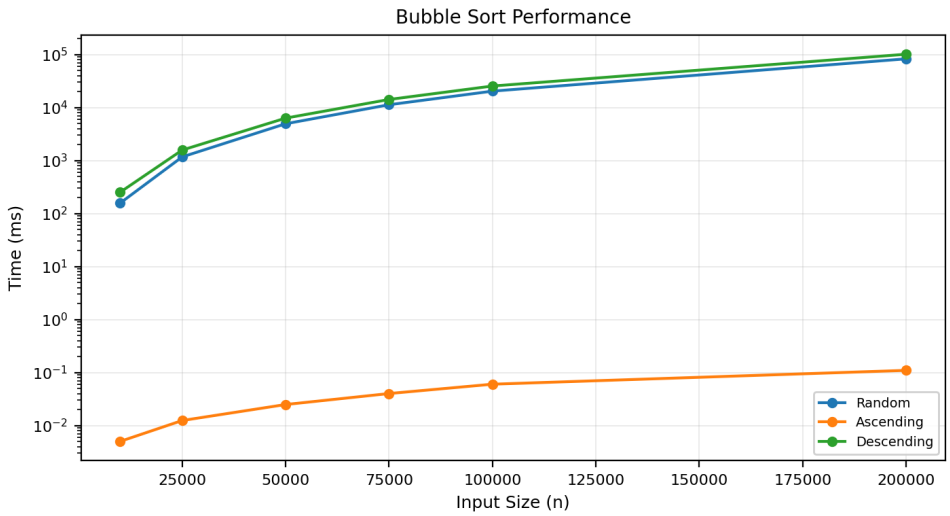


Figure 4: Bubble Sort Performance (logarithmic time axis).

4.2 Insertion Sort

Insertion Sort is efficient on already increasing input because each new element is already in its correct position and no shifting is needed. Its average time on increasing input is only 0.0075 ms at 10,000 elements and 0.1740 ms at 200,000 elements, a factor of 23.2 for a 20-fold increase in size. On random input the time grows from 9.2889 ms to 3,591.4033 ms (a factor of 386.6), and on decreasing input from 17.7933 ms to 6,814.2933 ms (a factor of 383.0), both close to the factor of 400 expected for quadratic growth.

The decreasing case is roughly twice as slow as the random case (between 1.90 and 2.09 times, depending on the size), which agrees with theory: on random data each element is shifted past about half of the sorted prefix on average, while on reversed data it is shifted past all of it. The comparison counts show the same factor of two: at 10,000 elements random input needs 25,053,443 comparisons and decreasing input needs 49,995,000.

Insertion Sort is also much faster than Bubble Sort on random input, although both are quadratic: Bubble Sort takes between 17.0 and 24.0 times as long, depending on the size,

because Insertion Sort performs far fewer data movements than the repeated swaps of Bubble Sort. On increasing input the order reverses slightly, and Bubble Sort is the faster of the two at every size.

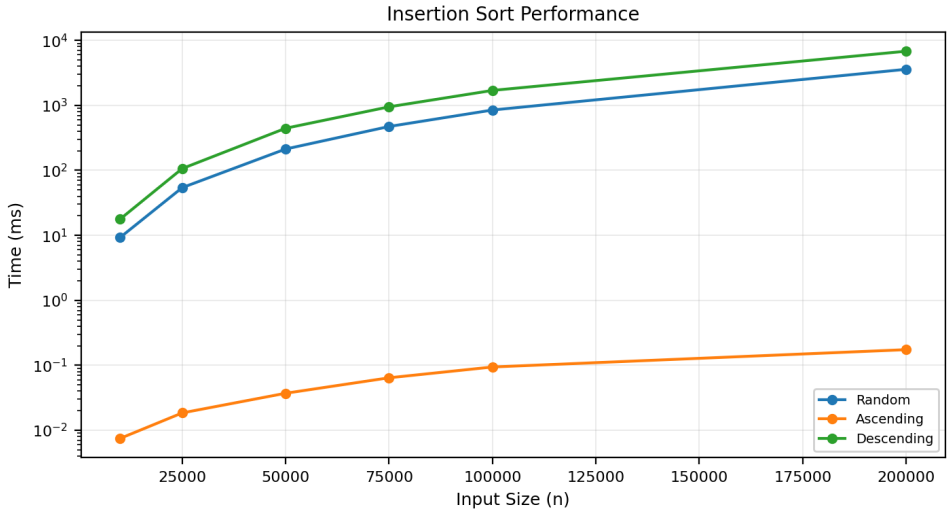


Figure 5: Insertion Sort Performance (logarithmic time axis).

4.3 Merge Sort

Merge Sort consistently shows $O(n \log n)$-type growth across all the tested input orders. Its measured time varies with the input order because of implementation and constant-factor effects, but the growth remains far more stable than that of the quadratic algorithms. At 500,000 elements, the averages are 39.2953 ms for random input, 9.4729 ms for increasing input, and 8.9919 ms for decreasing input. Random input is the slowest and decreasing input the fastest at every recorded size.

When the input grows from 10,000 to 500,000 elements (a factor of 50), the random-input time grows from 0.6057 ms to 39.2953 ms, a factor of 64.9, which is in line with the factor of about 71.2 predicted by $n \log n$ growth. Ordered input needs fewer comparisons because one half is exhausted early during merging: at 500,000 elements random input needs 8,838,089 comparisons, against 4,692,496 for increasing input and 4,783,216 for decreasing input.

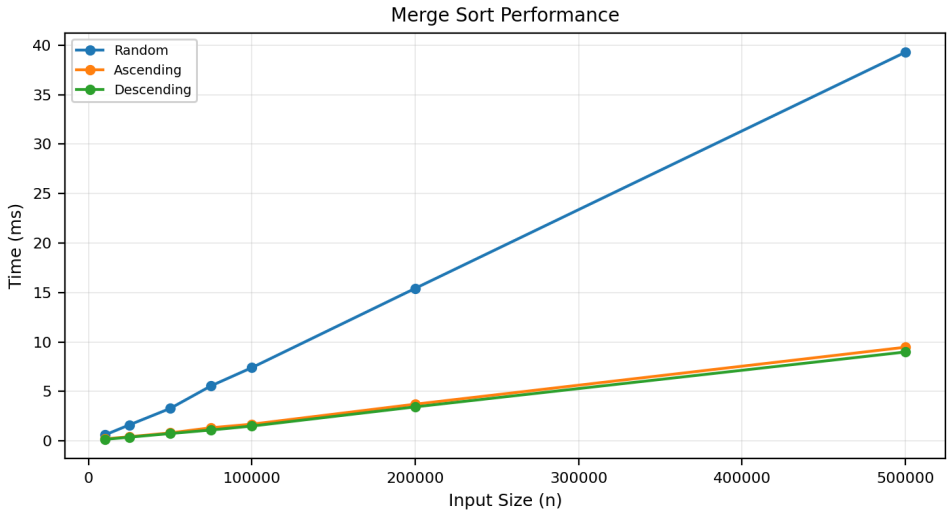


Figure 6: Merge Sort Performance.

4.4 Quick Sort

Quick Sort is highly dependent on the pivot strategy. The first-element pivot is fast on random input but becomes quadratic on ordered input. The random pivot keeps the measured times small for all three input types. Median-of-three is particularly fast on increasing input, but its decreasing-input results grow quadratically, so the implementation exhibits an unfavorable partition pattern for that case. Figure 7 compares the three versions on random input, where they are close to one another and all grow almost linearly on the plotted scale: from 10,000 to 500,000 elements the first-pivot time grows by a factor of 65.2, the random-pivot time by 59.3, and the median-of-three time by 60.7.



Figure 7: Quick Sort Performance on Random Input.

4.5 Heap Sort

Heap Sort has an $O(n \log n)$ worst-case bound and is not expected to become quadratic because of the input order. The measurements show stable growth. At 500,000 elements the average times are 48.8789 ms for random input, 26.4450 ms for increasing input, and 28.4229 ms for decreasing input. Random input is the slowest at every recorded size, taking about 1.85 and 1.72 times as long as increasing and decreasing input at 500,000 elements.

The comparison counts at that size are almost the same for the three orders (17,397,107 for random, 17,837,785 for increasing, and 16,977,997 for decreasing), so the difference in time is not explained by the number of comparisons. Heap operations access memory at widely separated positions, and on random data the comparison outcomes are also less predictable, so cache and branch effects are likely contributors. The time on random input grows by a factor of 69.4 between 10,000 and 500,000 elements. The increasing-input mean at 100,000 elements (6.8758 ms) is raised by one slow round, as noted in Section 2.3, which explains the small bump in Figure 8.

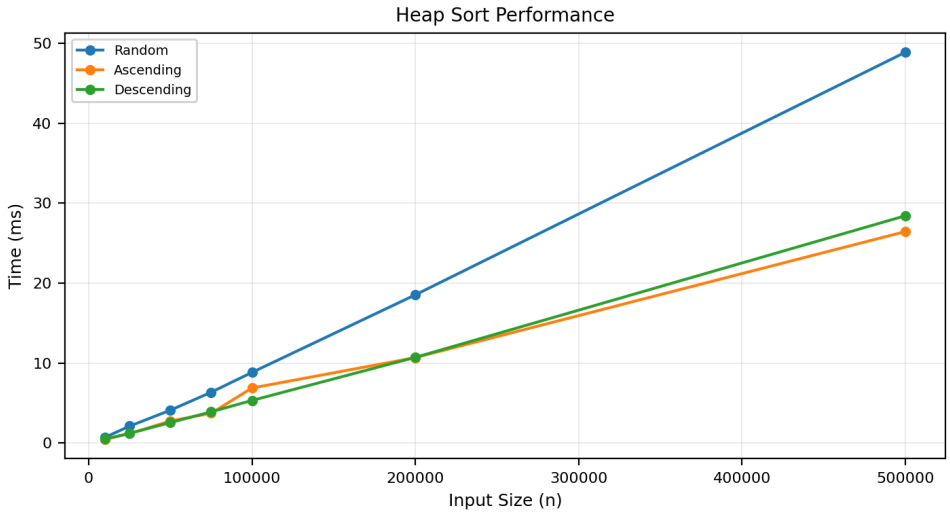


Figure 8: Heap Sort Performance.

4.6 Radix Sort

Radix Sort does not use element-to-element comparisons; in the recorded data its comparison count is zero for every entry. Its measured time is also relatively stable across input orders. At 500,000 elements the average time is 11.7244 ms for random input, 11.1723 ms for increasing input, and 11.2317 ms for decreasing input. Because each digit pass touches every element exactly once regardless of order, this insensitivity to the input is expected. The time on random input grows from 0.2253 ms to 11.7244 ms as the size grows by a factor of 50, a factor of 52.0, which is close to linear scaling.

The curve is not perfectly smooth at 100,000 elements, where the times (2.8138 ms on random input) are higher than a straight-line trend through the neighboring sizes would

suggest. The three rounds at this size vary considerably (see Section 2.3), so this point should be read with caution.

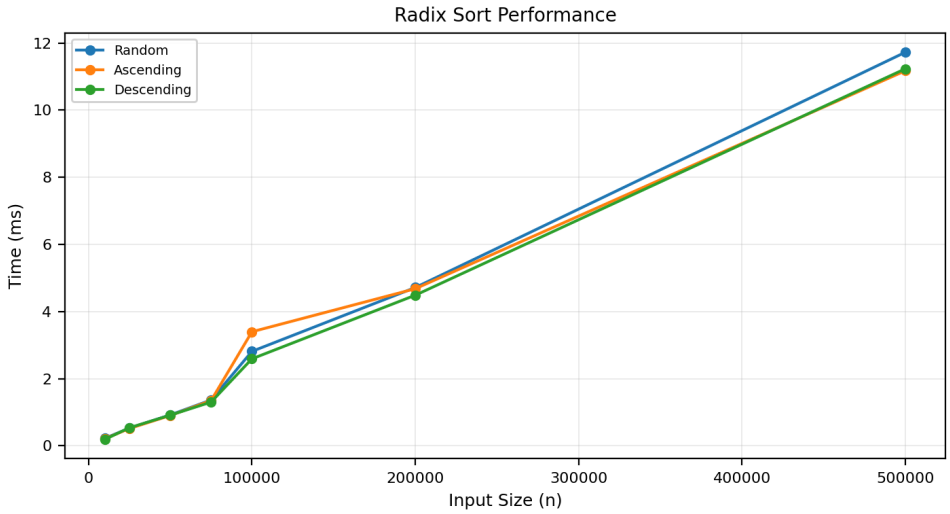


Figure 9: Radix Sort Performance.

5 Comparison of Sorting Algorithms

This section compares the algorithms as a group. The best-case graph uses the smallest measured time among the recorded input types at each size, the worst-case graph uses the largest measured time among the random, increasing, and decreasing inputs, and the average-case graph uses random input. All three graphs use a logarithmic time axis, and the legends are placed to the right of the plots. Curves for Bubble Sort and Insertion Sort stop at 200,000 elements, and the first-pivot curve in the worst-case graph stops at 75,000 elements, because no larger measurements exist.

5.1 Best Case Running Time

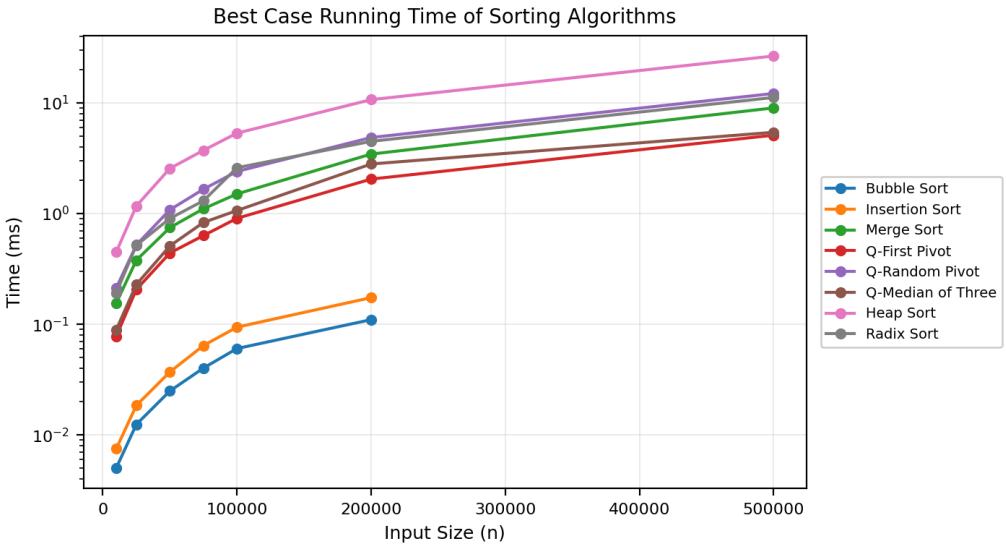


Figure 10: Best Case Running Time of the Sorting Algorithms as a Function of Input Size.

Figure 10 shows why simple algorithms can appear very fast on favorable input. Bubble Sort and Insertion Sort have the lowest best-case times of all algorithms at every size where they were measured: 0.1101 ms and 0.1740 ms at 200,000 elements, against 2.0469 ms for the next fastest, first-pivot Quick Sort on balanced input. This advantage exists only because the input is already sorted and the implementations perform a single linear pass, and it says nothing about their behavior on other inputs. At 500,000 elements, where only the efficient algorithms were measured, the lowest best-case time belongs to first-pivot Quick Sort on balanced input (5.1010 ms), followed by median-of-three Quick Sort on increasing input (5.4151 ms), Merge Sort (8.9919 ms), Radix Sort (11.1723 ms), random-pivot Quick Sort (12.1330 ms), and Heap Sort (26.4450 ms).

5.2 Worst Case Running Time

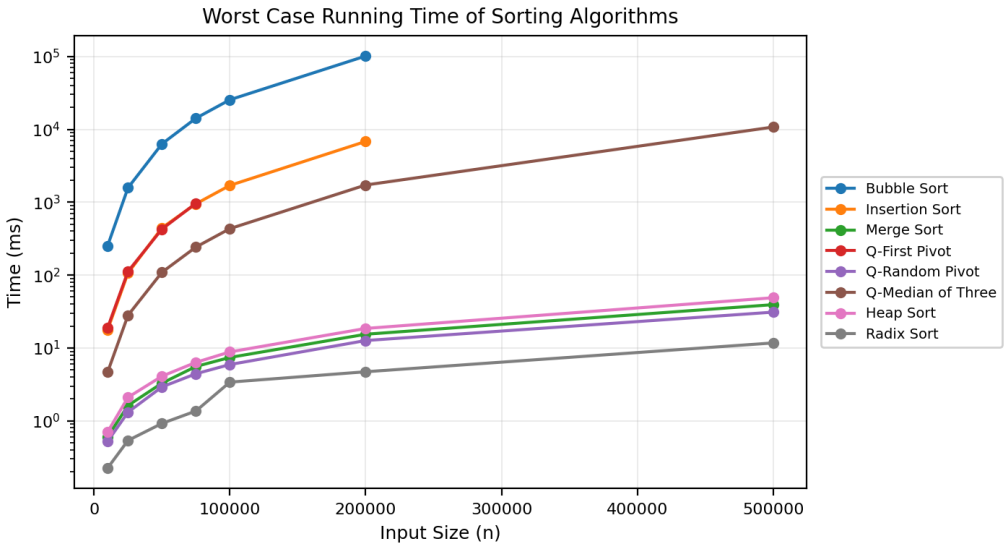


Figure 11: Worst Case Running Time of the Sorting Algorithms as a Function of Input Size.

The worst-case measurements clearly separate the algorithms with quadratic behavior from those with near-$O(n \log n)$ or linear behavior. At 10,000 elements the worst case of Bubble Sort (252.6447 ms) is already about 417 times that of Merge Sort (0.6057 ms). At 200,000 elements Bubble Sort needs 101,244.0000 ms and Insertion Sort 6,814.2933 ms, while median-of-three Quick Sort needs 1,721.4500 ms, against 15.4272 ms for Merge Sort, 18.5324 ms for Heap Sort, 12.6475 ms for random-pivot Quick Sort, and 4.7157 ms for Radix Sort.

First-pivot Quick Sort behaves like a quadratic algorithm on ordered data. At 75,000 elements its worst case (965.2950 ms) is about the same as that of Insertion Sort (949.9603 ms), and at each of the four sizes where both were measured the two differ by less than 7%. At 500,000 elements median-of-three Quick Sort reaches about 10.741 seconds, while Merge Sort (39.2953 ms), Heap Sort (48.8789 ms), random-pivot Quick Sort (31.0128 ms), and Radix Sort (11.7244 ms) remain far lower.

5.3 Average Case Running Time

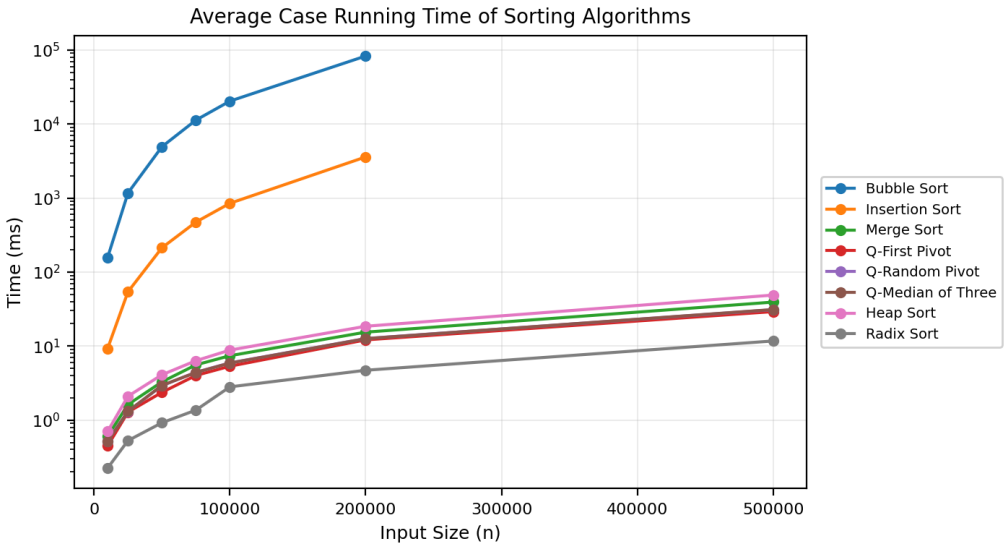


Figure 12: Average Case Running Time of the Sorting Algorithms on Random Input.

For randomly generated input, Radix Sort is the fastest algorithm at every recorded size. At 500,000 elements its average is 11.7244 ms. First-pivot Quick Sort is next at 29.2294 ms, followed by random-pivot Quick Sort at 31.0128 ms and median-of-three at 31.1902 ms. Merge Sort (39.2953 ms) and Heap Sort (48.8789 ms) are slower, and Merge Sort is faster than Heap Sort at every size. Bubble Sort and Insertion Sort are not available at 500,000 elements in the data; at 200,000 elements they take 82,874.8667 ms and 3,591.4033 ms, about 17,574 and 762 times the 4.7157 ms of Radix Sort.

5.4 Conclusion

The experimental analysis shows that there is no single best sorting algorithm for every situation. The input order, the pivot selection, and implementation details all affect the measured time.

- **Best-case performance:** Bubble Sort and Insertion Sort are the fastest of all on already sorted input, because their implementations need only a single linear pass. Among the efficient algorithms, first-pivot Quick Sort on a balanced input is the fastest.
- **Worst-case performance:** First-pivot Quick Sort shows the clearest quadratic growth on ordered input, matching Insertion Sort at the sizes where both were measured. The median-of-three implementation used here also shows quadratic deterioration on decreasing input.
- **Average-case performance:** On random data, Radix Sort is the fastest at every size, and the Quick Sort versions also scale well.

- **Reliable choices:** Merge Sort and Heap Sort maintain predictable $O(n \log n)$-type growth for every input order, and random-pivot Quick Sort provides strong protection against the ordered-input behavior seen with first-pivot Quick Sort.

In summary, the experimental evidence supports choosing an algorithm according to the expected data distribution, rather than relying only on its asymptotic label.

6 Correlation of Comparisons with Execution Time

Besides time, the number of comparisons made by each algorithm was recorded. This section examines how well the comparison count explains the running time. Where the correlation between the two is close to 1, counting comparisons is a good machine-independent predictor of speed. Where it is lower, other costs such as memory access, data movement, and branching make a larger contribution. The correlation coefficients quoted below are Pearson coefficients computed over all individual recorded runs of the algorithm (all input types and sizes). The time per comparison in the figures is the mean time divided by the mean comparison count, computed separately for each input type and size.

6.1 Insertion Sort

In the best case, Insertion Sort performs exactly $n - 1$ comparisons (9,999 at 10,000 elements and 199,999 at 200,000 elements). The correlation between time and comparison count across the 54 recorded runs is 0.9996, a strong experimental match between comparison count and execution time. The time per comparison is nearly constant on random input (3.38×10^{-7} to 3.71×10^{-7} ms) and on decreasing input (3.38×10^{-7} to 3.56×10^{-7} ms). On increasing input it is higher (7.40×10^{-7} to 9.42×10^{-7} ms), because the total time is very small and each comparison carries relatively more fixed per-element cost.

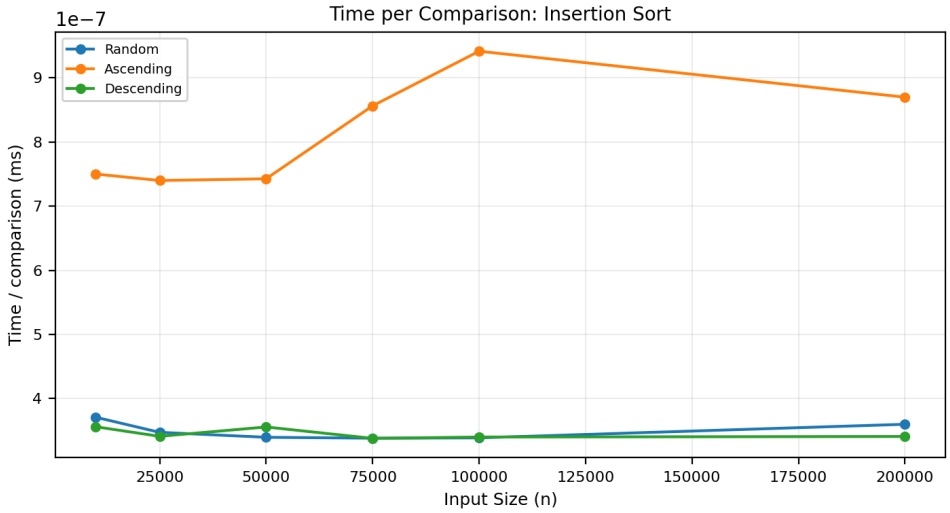


Figure 13: Time per Comparison for Insertion Sort.

6.2 Bubble Sort (Optimized)

Bubble Sort also makes exactly $n-1$ comparisons when the input is already sorted. The measured correlation between time and comparisons is 0.9935 across the 54 recorded runs. The time per comparison is about 4.99×10^{-7} to 6.04×10^{-7} ms on increasing input, 3.15×10^{-6} to 4.14×10^{-6} ms on random input (rising slowly with size), and almost constant on decreasing input (5.04×10^{-6} to 5.09×10^{-6} ms). Random and decreasing input make almost the same number of comparisons at a given size, yet decreasing input takes longer (for example 252.6447 ms against 157.5460 ms at 10,000 elements), so here the swaps contribute to the time in addition to the comparisons.

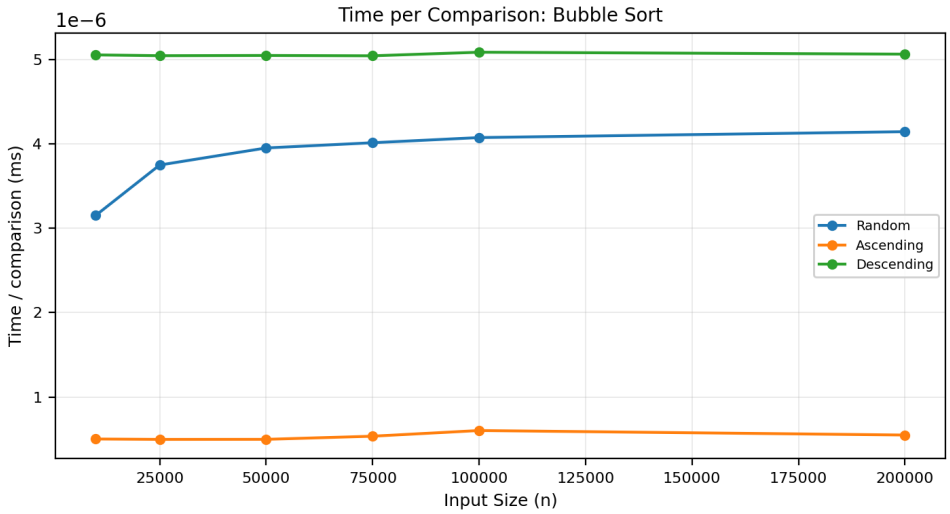


Figure 14: Time per Comparison for Bubble Sort.

6.3 Heap Sort

Heap Sort performs many comparisons during heap construction and during the repeated extractions. The measured correlation between time and comparisons is 0.9342 across 63 runs. Small variations are expected because execution time also includes memory accesses, swaps, branching, and system-level effects. As Figure 15 shows, the time per comparison is highest on random input (2.81×10^{-6} to 3.21×10^{-6} ms) and lower on increasing input (1.48×10^{-6} to 2.21×10^{-6} ms) and decreasing input (1.67×10^{-6} to 2.21×10^{-6} ms), even though the three input orders need nearly the same number of comparisons. This is consistent with the observation in Section 4.5 that random input is slower for reasons other than the comparison count.

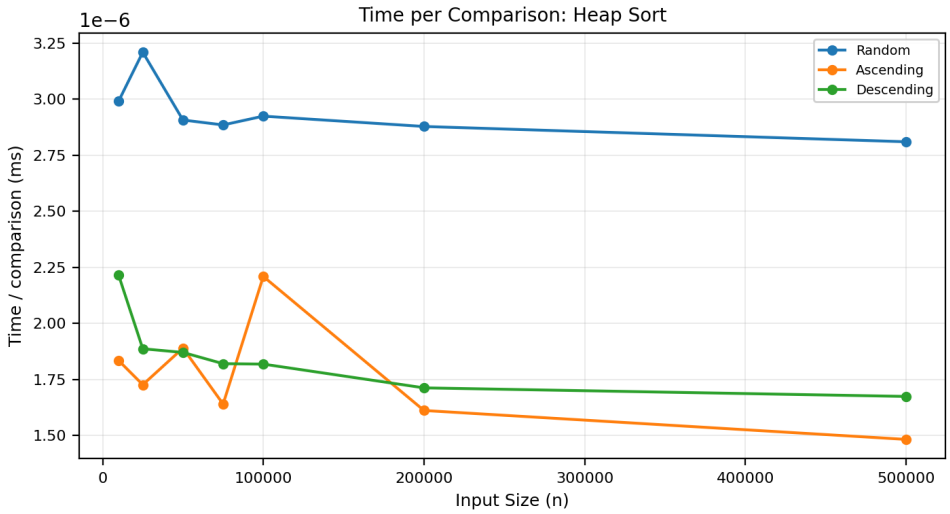


Figure 15: Time per Comparison for Heap Sort.

6.4 Merge Sort

Merge Sort makes a number of comparisons proportional to $n \log n$. The measured correlation between time and comparisons is 0.9279 across 63 runs, a strong but not perfect relationship. The deviations arise from memory access and the cost of moving elements in addition to the comparisons themselves. Ordered input needs about 1.9 times fewer comparisons than random input at 500,000 elements, but runs about 4.1 times faster, so comparisons alone do not capture the whole cost. The time per comparison is 4.45×10^{-6} to 5.03×10^{-6} ms on random input, 2.02×10^{-6} to 3.15×10^{-6} ms on increasing input, and 1.76×10^{-6} to 2.25×10^{-6} ms on decreasing input.

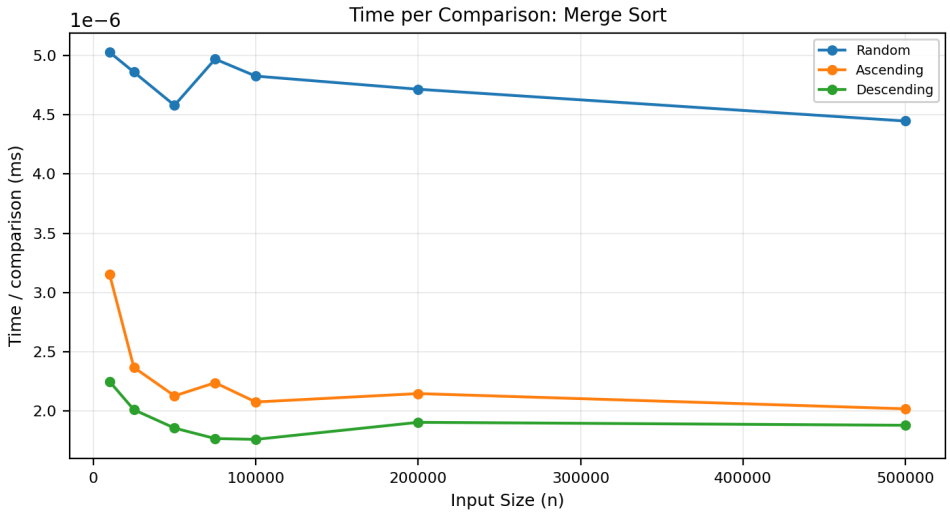


Figure 16: Time per Comparison for Merge Sort.

6.5 Quick Sort (Median-of-Three Pivot)

The median-of-three measurements show an almost perfect correlation between execution time and comparison count, 0.99999 across 63 runs. This is expected because the implementation performs a very large number of comparisons whenever its partitioning becomes unfavorable (31,250,250,002 at 500,000 decreasing elements), and those comparisons dominate the running time. The time per comparison is almost constant on decreasing input (3.44×10^{-7} to 3.73×10^{-7} ms), and higher on random input (3.21×10^{-6} to 3.95×10^{-6} ms) and on increasing input (6.39×10^{-7} to 8.93×10^{-7} ms). The result is a useful example of why counting comparisons helps to explain observed running time. For completeness, the correlations for the other two versions are 0.9754 for first-pivot Quick Sort and 0.8573 for random-pivot Quick Sort; the latter is the lowest of all algorithms. Random-pivot Quick Sort makes nearly the same number of comparisons on the three input orders (11,529,409 on random, 11,450,702 on increasing, and 11,749,285 on decreasing input at 500,000 elements, as means over three rounds), yet its random-input time is between 2.06 and 2.47 times that of the slower ordered input, so the comparison count explains less of its running time.

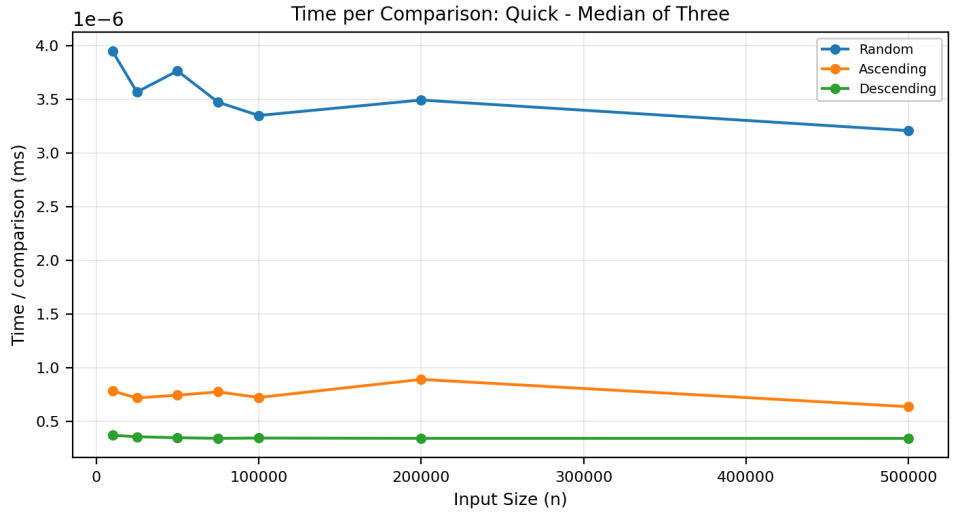


Figure 17: Time per Comparison for Quick Sort with Median-of-Three Pivot.

6.6 Radix Sort

Radix Sort does not rely on comparisons. Its comparison count is recorded as zero in the data, so time per comparison is not a meaningful metric. Instead, its running time depends mainly on the number of elements and the number of digit passes, and Figure 18 shows that it grows almost linearly with the input size for all three input orders.



Figure 18: Radix Sort Execution Time; Comparison Count is Zero.

6.7 Conclusion

For the comparison-based algorithms (Insertion, Bubble, Heap, Merge, and Quick Sort), execution time generally follows the number of comparisons. The correlation is strongest where the comparison count varies over orders of magnitude, as for median-of-three Quick Sort (0.99999), Insertion Sort (0.9996), and Bubble Sort (0.9935). It is lower for Heap Sort (0.9342), Merge Sort (0.9279), and random-pivot Quick Sort (0.8573), whose comparison counts follow an $n \log n$ pattern and whose time per comparison differs noticeably between input orders, so that memory access, data movement, and branch behavior make a larger share of the time. For small inputs, timing fluctuations can also be caused by system overhead and measurement resolution. In contrast, Radix Sort does not use comparisons: its measured comparison count is zero, while its execution time remains approximately linear in the input size for the tested integer data.

7 Execution Time Analysis

The following tables present the average execution time in milliseconds for random, increasing, and decreasing inputs. The values are averages over the three recorded rounds in the experimental data.

7.1 Bubble Sort

Input Size	Random	Increasing	Decreasing
10000	157.5460	0.0050	252.6447
25000	1171.3900	0.0125	1576.1867
50000	4938.6367	0.0250	6308.2933
75000	11285.7000	0.0402	14182.4000
100000	20369.3333	0.0604	25425.1000
200000	82874.8667	0.1101	101244.0000

Table 1: Average Execution Time for Bubble Sort

7.2 Insertion Sort

Input Size	Random	Increasing	Decreasing
10000	9.2889	0.0075	17.7933
25000	54.2226	0.0185	106.5607
50000	212.9403	0.0371	444.4657
75000	473.2533	0.0642	949.9603
100000	848.5377	0.0942	1699.3367
200000	3591.4033	0.1740	6814.2933

Table 2: Average Execution Time for Insertion Sort

Note: Bubble Sort and Insertion Sort have measurements only up to 200,000 elements in the recorded results. No larger values have been estimated or invented.

7.3 Merge Sort

Input Size	Random	Increasing	Decreasing
10000	0.6057	0.2038	0.1552
25000	1.6227	0.4232	0.3790
50000	3.2878	0.8140	0.7461
75000	5.5710	1.3312	1.1038
100000	7.4137	1.6921	1.5036
200000	15.4272	3.7149	3.4435
500000	39.2953	9.4729	8.9919

Table 3: Average Execution Time for Merge Sort

7.4 Heap Sort

Input Size	Random	Increasing	Decreasing
10000	0.7042	0.4483	0.5019
25000	2.1021	1.1699	1.1956
50000	4.0982	2.7500	2.5549
75000	6.3469	3.7197	3.8926
100000	8.8284	6.8758	5.3213
200000	18.5324	10.6788	10.7125
500000	48.8789	26.4450	28.4229

Table 4: Average Execution Time for Heap Sort

7.5 Radix Sort

Input Size	Random	Increasing	Decreasing
10000	0.2253	0.2109	0.1924
25000	0.5290	0.5166	0.5385
50000	0.9217	0.9030	0.9132
75000	1.3653	1.3493	1.3035
100000	2.8138	3.3982	2.5921
200000	4.7157	4.6803	4.4874
500000	11.7244	11.1723	11.2317

Table 5: Average Execution Time for Radix Sort

7.6 Quick Sort (Median-of-Three Pivot)

For Quick Sort, the table below uses the median-of-three version as the representative Quick Sort table, while Section 3 compares all three pivot strategies separately. The values are the actual measured averages and are not substituted from theory.

Input Size	Random	Increasing	Decreasing
10000	0.5140	0.0892	4.6651
25000	1.3160	0.2286	28.0543
50000	2.9464	0.5105	109.2890
75000	4.4043	0.8304	242.1840
100000	5.8978	1.0645	433.4407
200000	12.6001	2.8016	1721.4500
500000	31.1902	5.4151	10740.6667

Table 6: Average Execution Time for Quick Sort (Median-of-Three Pivot)

7.7 Quick Sort: Comparison of the Three Versions

The following compact table summarizes the three Quick Sort versions on random input. It allows a direct comparison of the measured average running time at every available input size.

Input Size	First Pivot	Random Pivot	Median of Three
10000	0.4485	0.5226	0.5140
25000	1.2838	1.3150	1.3160
50000	2.3737	2.9196	2.9464
75000	4.0119	4.4297	4.4043
100000	5.3448	5.9265	5.8978
200000	12.1308	12.6475	12.6001
500000	29.2294	31.0128	31.1902

Table 7: Average Random-Input Execution Time of Quick Sort Versions (ms)

The final comparison shows the main practical result of the experiment. For random input, first-pivot Quick Sort has the lowest measured time of the three at every listed size. However, the ordered-input experiments show that it is not robust to adverse input order. The random pivot gives the most stable behavior across input types, while median-of-three is excellent on increasing and random data but shows a large deterioration on decreasing data in this implementation.

7.8 Quick Sort (First Pivot)

The complete first-pivot measurements are given below, including the balanced input. A dash (—) marks a combination that was not run: increasing and decreasing input were recorded for the first pivot only up to 75,000 elements.

Input Size	Random	Increasing	Decreasing	Balanced
10000	0.4485	12.4075	18.9431	0.0779
25000	1.2838	73.9663	112.1590	0.2060
50000	2.3737	282.5863	427.3397	0.4402
75000	4.0119	642.8610	965.2950	0.6323
100000	5.3448	—	—	0.9051
200000	12.1308	—	—	2.0469
500000	29.2294	—	—	5.1010

Table 8: Average Execution Time for Quick Sort (First Pivot)

7.9 Quick Sort (Random Pivot)

Input Size	Random	Increasing	Decreasing
10000	0.5226	0.2122	0.2543
25000	1.3150	0.5208	0.5901
50000	2.9196	1.0812	1.1861
75000	4.4297	1.6571	1.7924
100000	5.9265	2.3987	2.5094
200000	12.6475	4.8583	5.2612
500000	31.0128	12.1330	13.6299

Table 9: Average Execution Time for Quick Sort (Random Pivot)

Overall Conclusion

This study compared six sorting algorithms, including three Quick Sort variants, on random, increasing, and decreasing integer arrays of up to 500,000 elements. The main findings are as follows.

1. **Asymptotic notation is a guide, not a verdict.** The quadratic algorithms are impractical at scale (Bubble Sort needs 101.244 seconds on 200,000 decreasing elements), but among the $O(n \log n)$ and linear methods the constant factors, memory behavior, and input order decided the ranking.
2. **Pivot selection is decisive for Quick Sort.** At 75,000 elements, first-pivot Quick Sort ranges from 0.6323 ms on the balanced input and 4.0119 ms on random input to 965.2950 ms on decreasing input. The random pivot is the most dependable of the three variants.
3. **A named strategy is not a guarantee.** The median-of-three implementation was very fast on increasing and random data, but grew quadratically on decreasing data (10.741 seconds at 500,000 elements), which suggests that its partitioning step should be inspected and corrected before it is relied upon.
4. **Merge Sort and Heap Sort are predictable.** Their running times are largely insensitive to the input order and grow in line with $n \log n$.
5. **Radix Sort is fastest on the tested random integers.** It is the fastest at every size, needs no comparisons, and scales almost linearly, at the cost of being limited to data that can be split into digits.
6. **Comparison counts explain time well for the comparison sorts.** The measured correlations range from 0.8573 to 0.99999, with the remaining variation attributable to memory access, data movement, and branching.

Limitations and Future Work

The conclusions should be read with the following limitations in mind. Bubble Sort and Insertion Sort were measured only up to 200,000 elements, and first-pivot Quick Sort on increasing and decreasing input only up to 75,000 elements, so their behavior at larger sizes is not covered by the data. Each result is an average of only three rounds, and a single slow round can visibly affect a mean, as in the cases noted in Section 2.3. The results reflect a single implementation and the single machine described in Section 2.2. Only integer data and one random distribution were used, and the balanced input is an artificially constructed best case for the first-pivot rule. Future work could extend the quadratic algorithms and the ordered first-pivot runs to larger sizes, increase the number of rounds, test further input patterns such as nearly sorted data and data with many duplicate values, examine the cause of the decreasing-input behavior of the median-of-three implementation, and compare the algorithms on additional hardware and compilers.

End of Report